

7. Project Documentation

7.1 Containerized Orchestration & Setup

The application components run within a single declarative Docker Compose configuration. Follow the guide below to deploy the runtime infrastructure:

Step 1: Environment Keys Configuration

Configure a settings file named `.env` in the project root directory and add the configuration keys:

```
# System Infrastructure Directives
DEBUG_STATUS=False
SECRET_ENCRYPTION_KEY=c3BsaXRfdGhlX2F0b21faGVscF9zZWN1cm10eV9maXJzdF9zdHJpbmc=

# Database Connection URIs
POSTGRES_DB_URI=postgresql+asyncpg://postgres_admin:SecurePass2026@db_cluster:5432/
network_assistant
REDIS_BROKER_URL=redis://redis_cache_node:6379/0

# Intelligence API Credentials
OPENAI_API_KEY=sk-proj-A1B2C3D4E5F6G7H8I9J0K1L2M3N4O5P6Q7R8S9T0
```

Step 2: Initialize Application Containers

Run the orchestration launch command to assemble the component containers and build the local network mesh:

```
# Build and launch microservice infrastructure containers
docker-compose up --build -d

# Execute data structure migrations across backend tables
docker-compose exec api_server alembic upgrade head
```

7.2 CI/CD Pipelines & Cloud Deployment

For cloud production setups, components are packaged using clean Helm configurations and deployed onto an AWS EKS managed Kubernetes cluster. External traffic is handled via an Ingress controller configured to enforce HTTPS communication policies. Production database storage uses an encrypted Amazon RDS PostgreSQL system set up for cross-zone replication, while operational cache management runs across an Amazon ElastiCache Redis setup.

Security Rule

All operational database environments must remain isolated within private subnets. External routing entries must communicate exclusively through security-hardened load balancer interfaces.